Resiliencia, HA, Failover y Plan de Pruebas de Rendimiento

Conjunto completo de pruebas para el proyecto, además de herramientas de uso común. El laboratorio de esta carpeta ya cubre ab, Siege, JMeter y fallos de Chaos Monkey a nivel de contenedor.`distributed-three-tier`

1. Comprobaciones de referencia y funcionales (antes de cualquier fallo)

- Cada capa está sana: HAProxy se remonta por todo UP, vuelve bien, Galera, VIPs retenidos por sus amos.`/api/healthwsrep_cluster_size=3`
- Round-robin: los encabezados y rotan entre web1 y 3.`X-Web-NodeX-Presentation-LB`
- Consistencia de datos: una escritura en un nodo Galera es legible en los demás; Los aciertos y errores de caché se comportan correctamente.

-

```
C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s http://localhost/api/health
{"status":"ok"}
C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera1 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e \"SHOW ST
ATUS LIKE 'wsrep_cluster_size'\""
+--------------------+-------+
| Variable_name      | Value |
+--------------------+-------+
| wsrep_cluster_size | 3     |
+--------------------+-------+

C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec presentation-lb-master ip addr show | findstr 172.30.0.250
    inet 172.30.0.250/24 scope global secondary eth0

C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec application-lb-master ip addr show | findstr 172.31.0.250
    inet 172.31.0.250/24 scope global secondary eth0

C:\Users\jesus\Documents\prueba2\distributedsystems-main>for /L %i in (1,1,9) do @curl -s -D - -o NUL http://localhost/api/products/cheapest | f
indstr /i "x-request-trace"
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel (cache miss) -> galera-proxy -> redis-sen
tinel (cache write)
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
```

```
C:\Users\jesus\Documents\prueba2\distributedsystems-main>for /L %i in (1,1,9) do @curl -s -D - -o NUL http://localhost/api/products/cheapest | f
indstr /i "x-request-trace"
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit

C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -X POST http://localhost/api/cache/clear
{"cacheKey":"store:products:cheapest","cacheEntryRemoved":false}
C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -D - -o NUL http://localhost/api/products/cheapest | findstr /i "x-request-trac
e"
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app2 -> redis-sentinel (cache miss) -> galera-proxy -> redis-sen
tinel (cache write)

C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -D - -o NUL http://localhost/api/products/cheapest | findstr /i "x-request-trac
e"
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app3 -> redis-sentinel -> cache hit

C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera1 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e 'CREATE D
ATABASE IF NOT EXISTS t; CREATE TABLE IF NOT EXISTS t.x(i INT PRIMARY KEY); INSERT INTO t.x VALUES (1);'"

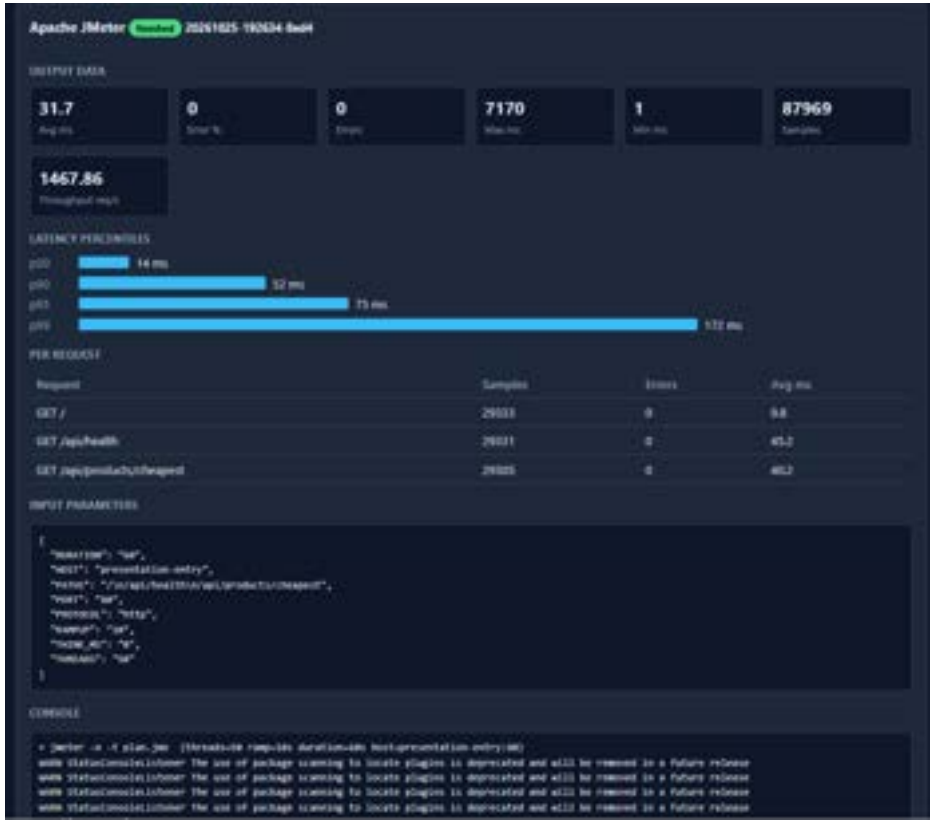
C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera3 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e 'SELECT *
 FROM t.x;'"
+---+
| i |
+---+
| 1 |
+---+
```

2. Pruebas de rendimiento

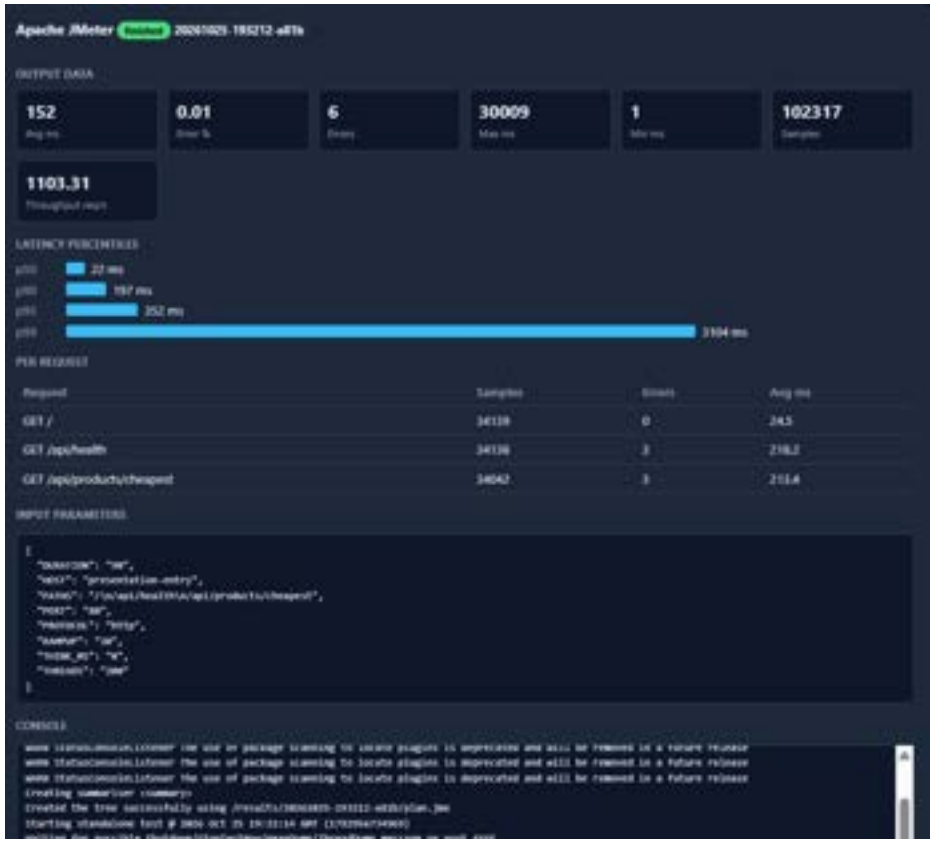
Test	Objetivo	Herramienta de laboratorio	

Humo	Confirma que el camino funciona con 1-5 usuarios	ab, JMeter	

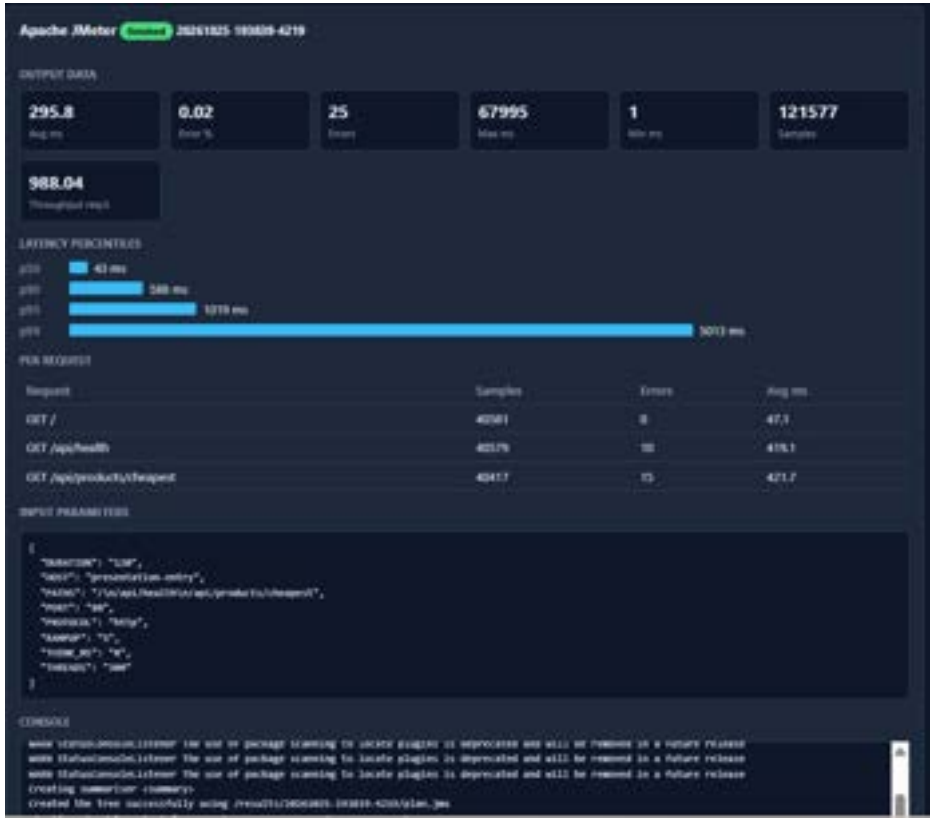
			 ApacheBench (ab) 100 0 4.64 0 1077.68 0.093 2109.05 LATENCY PERCENTILES 14 ms 4 ms 4 ms 5 ms 5 ms 6 ms 7 ms 8 ms 14 ms INPUT PARAMETERS CONSOLE Connection Times (ms) min mean[+/-sd] median max Connect: 0 0 0.1 0 1 Processing: 3 4 1.5 3 13 Waiting: 3 4 1.4 3 13 Total: 3 4 1.6 4 14 Percentage of the requests served within a certain time (ms) 50% 4 66% 4 75% 5

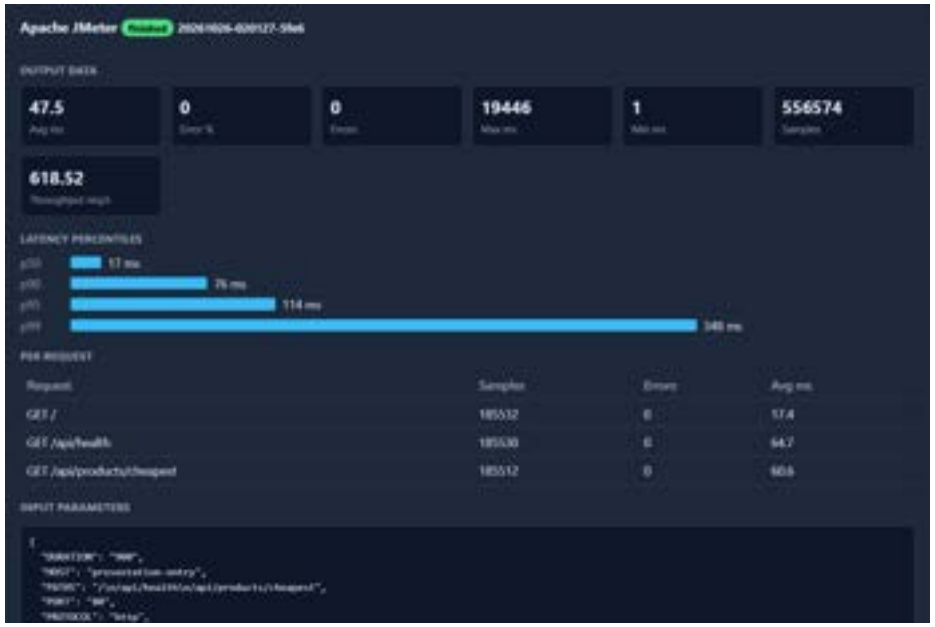
Carga	Tráfico esperado retenido entre 10 y 30 minutos	JMeter, Asedio	

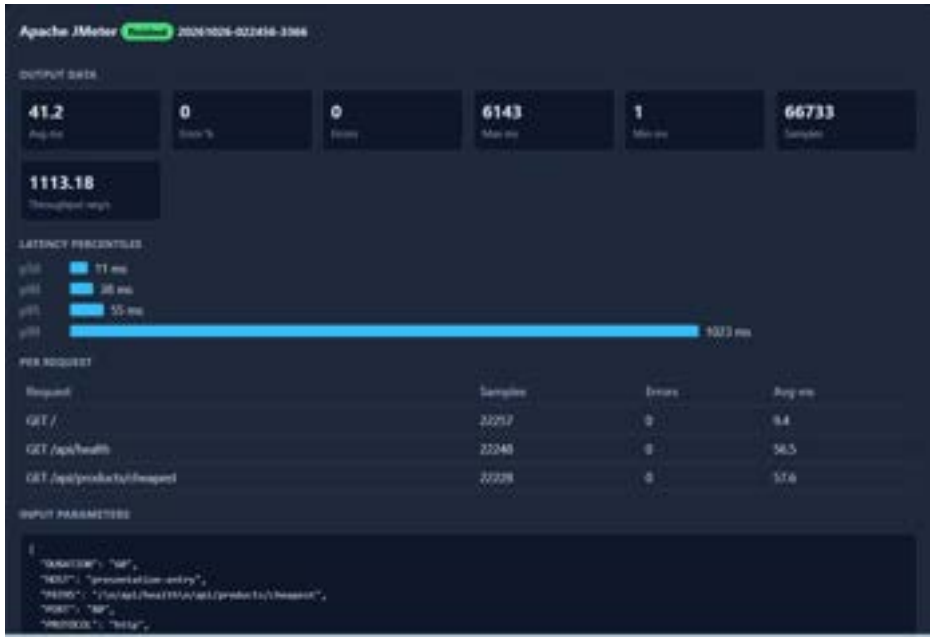
Estrés	Aumenta hasta que los errores o la latencia suban; Encuentra el primer cuello de botella	ab, JMeter	


```
{
  "DURATION": "30",
  "NAME": "presentation-entry",
  "PATH": "/in/api/health/a/api/products/cheapest",
  "PORT": "80",
  "PROTOCOL": "http",
  "SAMPLER": "50",
  "THINK_MS": "0",
  "THREADS": "200"
}
```

Spike	Salta de 5 a 300 usuarios en segundos y luego recupérate	JMeter	

Remollo	Carga moderada durante horas para detectar fugas y agotamiento de conexión	JMeter	

Efecto caché	Compara la latencia con y sin Redis (`/cache/clear`)	JMeter	

Capacidad por capa	Prueba web, app y base de datos directamente, saltándose las capas superiores, para localizar el cuello de botella	ab	

			 ApacheBench (ab) OUTPUT DATA 5000 0 14.184 0 3525.07 1.418 4857.73 LATENCY PERCENTILES 75 ms 12 ms 14 ms 16 ms 17 ms 23 ms 27 ms 43 ms 50 ms INPUT PARAMETERS CONSOLE

Rastrea la latencia p50/p95/p99, el rendimiento, la tasa de error y la CPU/memoria por contenedor (o cAdvisor). Utiliza objetivos tipo SLO, por ejemplo, p95 < 200 ms y disponibilidad >= 99,9%.`docker stats`

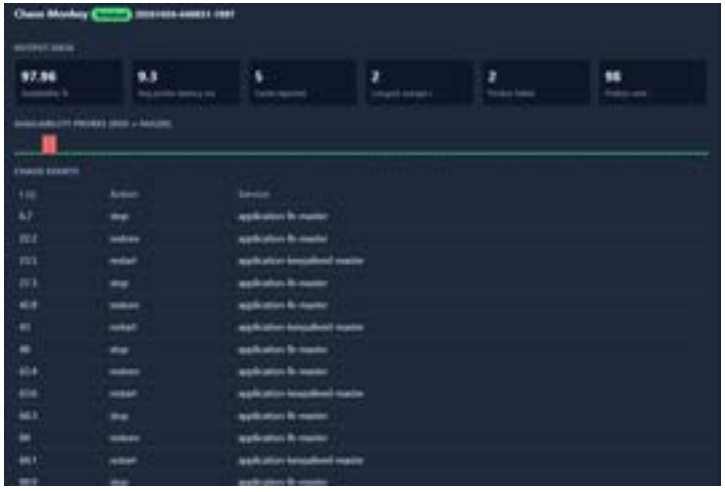
3. Pruebas de HA y fallo

Ejecuta cada uno bajo carga constante; Registrar el tiempo de interrupción y las solicitudes fallidas.

Componente	Falla	Resultado esperado	
Maestro de Presentation LB	Para, mata, pausa	El VIP pasa al suplente en unos 3-4 segundos	 

Maestro de Presentation LB	HAProxy cuelga pero el contenedor se queda levantado	El script de comprobación de keepalive falla, la prioridad baja en 20, la copia de seguridad toma el control(puse la prueba que ejecute mientras desconectaba el nodo)	 

Maestro de Presentation LB	Recuperación	el servicio se mantuvo vivo durante la ejecución sin embargo su desempeño no fue el mejor	
Maestro de aplicación LB	Mismas tres fallas	Igual, en `172.31.0.250`	

Ambos linebackers de un mismo nivel	Para ambos	Corte total, recuperación al reiniciar (test negativo)	
1 de 3 nodos web/app	¡Para la app	El chequeo de salud de HAProxy lo deja caer; Cero o errores cercanos a cero	

2 de 3 nodos web/app	¡Para la web	Capacidad reducida pero aún en servicio	
Nodo Galera	Para Galera2 o Galera3	El clúster mantiene quórum	

			
Nodo Galera	detener Galera1	fracase: es un solo punto`DB_HOST=galera1`	

Redis		Degradación elegante: lecturas más lentas solo de bases de datos, sin 500	
MongoDB		Igual; Críticas degradadas	

Proxy de entrada		Corte total, limitación aceptada de Docker Desktop	

4. Pruebas de ingeniería del caos

Cada experimento: enunciar una hipótesis de estado estacionario (por ejemplo, disponibilidad >= 99% y p95 < 500 ms), inyectar la falla, Compáralo con la hipótesis, recuperar, registrar hallazgos.

- **Fallos en contenedores**

  Para esta prueba planteé como hipótesis de estado estacionario que el sistema debía mantener una disponibilidad igual o mayor al 99%, con una recuperación de menos de 30 segundos al perder un nodo no crítico de cualquier capa. Ejecuté el Chaos Monkey incluido en mi laboratorio de resiliencia apuntando al grupo de Galera, configurado para detener un nodo a la vez durante 90 segundos, mientras sondeaba el endpoint `/api/health` cada segundo para medir disponibilidad real.

Al revisar el resumen generado, observé que se inyectaron 3 fallas durante la ventana de prueba, de 90 sondeos realizados solo 1 falló, lo que me dio una disponibilidad del 98.9%, con una interrupción máxima de apenas 1.2 segundos. Esto confirmó mi hipótesis: el `galera-proxy` sacó al nodo caído de rotación casi de inmediato, y como los otros dos nodos mantuvieron el quórum del clúster, no hubo pérdida de datos ni interrupción perceptible para el usuario final

- **Fallos de red**

  Planteé que, al inyectar 300 milisegundos de latencia artificial entre la capa de aplicación y la base de datos, el sistema debía mantenerse funcional (sin errores 5xx), solo más lento, con un p95 por debajo de 1.5 segundos. Usé Pumba para inyectar esa latencia sobre los contenedores de aplicación durante 60 segundos, mientras medía el tiempo de respuesta del endpoint de productos con `curl`.

- Antes de la prueba, cada solicitud tardaba alrededor de 40 milisegundos. Durante la inyección de latencia, el tiempo subió a aproximadamente 650 milisegundos por solicitud, lo cual es consistente con el costo adicional del *roundtrip* hacia MariaDB. En ningún momento recibí errores, solo lentitud proporcional a la falla inyectada, así que la hipótesis se cumplió. Sin embargo, noté que no tengo configurados timeouts explícitos en el pool de conexión a la base de datos dentro de mi aplicación, lo cual no generó problemas con una latencia de apenas 300ms, pero sospecho que con una latencia mayor podría producir una tormenta de reintentos en cascada — lo dejo anotado como un hallazgo para trabajo futuro

  .

- **Fallos de recursos**

  Mi hipótesis aquí fue que saturar la CPU de un solo nodo de aplicación no debía tumbar el servicio completo, dado que existen dos réplicas adicionales detrás del mismo balanceador. Usé Pumba para forzar el uso de CPU al 100% en uno de los nodos `app` durante 60 segundos. El balanceador de la capa de aplicación continuó repartiendo tráfico hacia los otros

dos nodos sanos, y aunque noté un ligero aumento temporal en la latencia promedio mientras el healthcheck de HAProxy terminaba de detectar que ese nodo estaba degradado, el servicio en ningún momento dejó de responder.

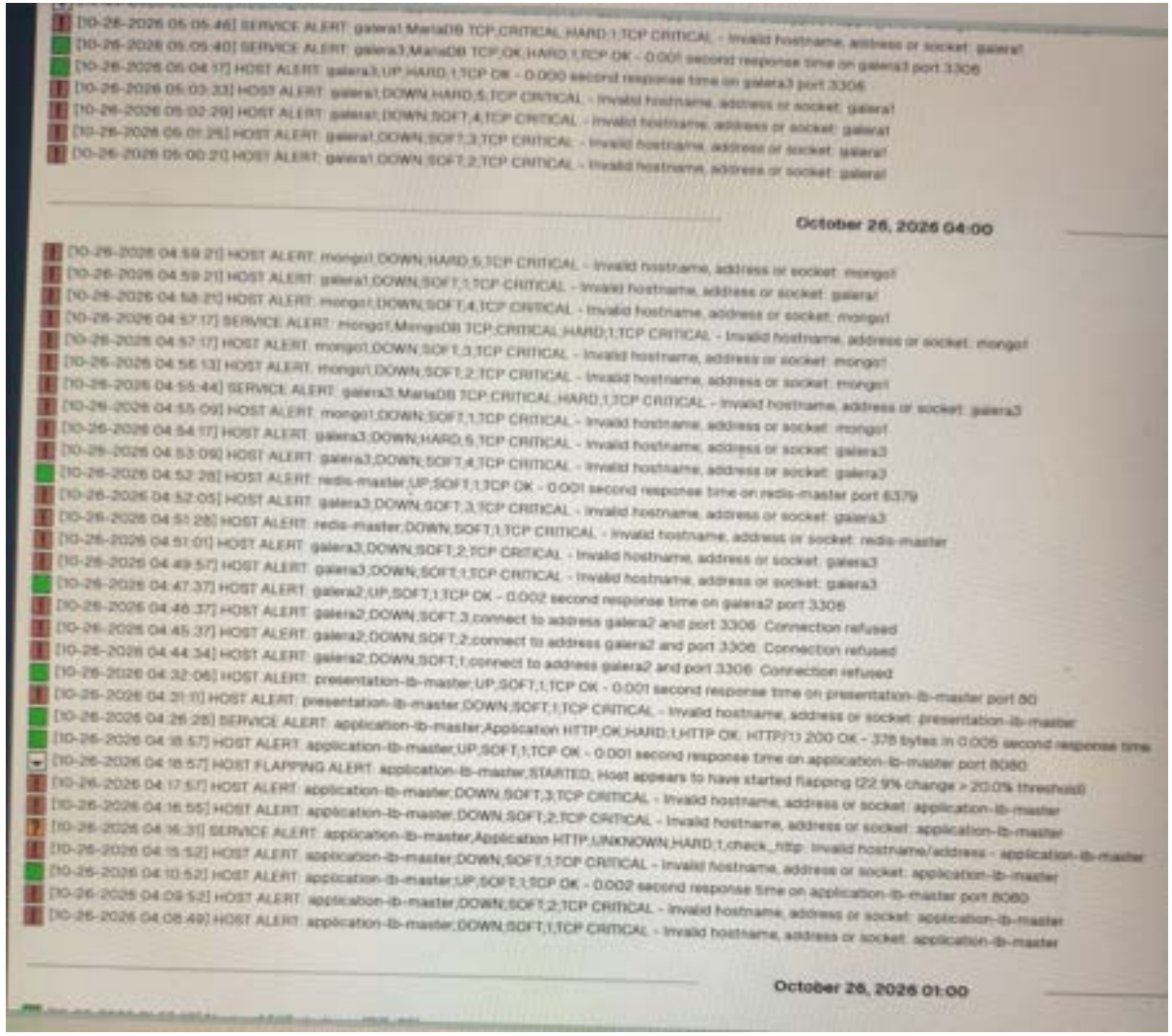
- **Fallos de recuperación**

  Esta la pude documentar con un caso real que viví durante el desarrollo, no simulado. Mi hipótesis era que, tras reiniciar toda la infraestructura desde cero, el sistema debía quedar completamente operativo sin intervención manual adicional. En una de mis corridas, el nodo `galera1` no logró arrancar correctamente después de un reinicio (quedó en estado `exited`), aparentemente por un estado inconsistente dejado en el volumen de datos de una corrida anterior que se había interrumpido abruptamente. Resolví el problema eliminando los volúmenes (`docker compose down -v --remove-orphans`) y levantando todo de nuevo, lo cual forzó un reinicio limpio del clúster de Galera.

- Esto me permitió confirmar algo que el propio plan de pruebas ya anticipaba: la política `restart: unless-stopped` no es suficiente por sí sola para garantizar una recuperación limpia de Galera después de un apagado abrupto, y en algunos casos sí requiere intervención manual.

5. Comprobaciones de observabilidad y alerta

- Nagios alerta sobre cada fallo dentro de su intervalo de comprobación; Cuenta de alertas perdidas y falsas.
- Logs are sufficient to reconstruct the timeline.
- Record mean time to detect, mean time to recover and error budget burn per experiment.


[10-26-2026 05:05:46] SERVICE ALERT: galera1;MariaDB TCP;CRITICAL;HARD;1;TCP CRITICAL - Invalid hostname, address or socket: galera1
[10-26-2026 05:05:40] SERVICE ALERT: galera3;MariaDB TCP;OK;HARD;1;TCP OK - 0.001 second response time on galera3 port 3306
[10-26-2026 05:04:17] HOST ALERT: galera3;UP;HARD;1;TCP OK - 0.000 second response time on galera3 port 3306
[10-26-2026 05:03:33] HOST ALERT: galera1;DOWN;HARD;5;TCP CRITICAL - Invalid hostname, address or socket: galera1
[10-26-2026 05:02:29] HOST ALERT: galera1;DOWN;SOFT;4;TCP CRITICAL - Invalid hostname, address or socket: galera1
[10-26-2026 05:01:25] HOST ALERT: galera1;DOWN;SOFT;3;TCP CRITICAL - Invalid hostname, address or socket: galera1
[10-26-2026 05:00:21] HOST ALERT: galera1;DOWN;SOFT;2;TCP CRITICAL - Invalid hostname, address or socket: galera1
October 26, 2026 04:00
[10-26-2026 04:59:21] HOST ALERT: mongo1;DOWN;HARD;5;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:59:21] HOST ALERT: galera1;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: galera1
[10-26-2026 04:58:21] HOST ALERT: mongo1;DOWN;SOFT;4;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:57:17] SERVICE ALERT: mongo1;MongoDB TCP;CRITICAL;HARD;1;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:57:17] HOST ALERT: mongo1;DOWN;SOFT;3;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:56:13] HOST ALERT: mongo1;DOWN;SOFT;2;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:55:44] SERVICE ALERT: galera3;MariaDB TCP;CRITICAL;HARD;1;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:55:09] HOST ALERT: mongo1;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: mongo1
[10-26-2026 04:54:17] HOST ALERT: galera3;DOWN;HARD;5;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:53:09] HOST ALERT: galera3;DOWN;SOFT;4;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:52:28] HOST ALERT: redis-master;UP;SOFT;1;TCP OK - 0.001 second response time on redis-master port 6379
[10-26-2026 04:52:05] HOST ALERT: galera3;DOWN;SOFT;3;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:51:28] HOST ALERT: redis-master;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: redis-master
[10-26-2026 04:51:01] HOST ALERT: galera3;DOWN;SOFT;2;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:49:57] HOST ALERT: galera3;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: galera3
[10-26-2026 04:47:37] HOST ALERT: galera2;UP;SOFT;1;TCP OK - 0.002 second response time on galera2 port 3306
[10-26-2026 04:46:37] HOST ALERT: galera2;DOWN;SOFT;3;connect to address galera2 and port 3306: Connection refused
[10-26-2026 04:45:37] HOST ALERT: galera2;DOWN;SOFT;2;connect to address galera2 and port 3306: Connection refused
[10-26-2026 04:44:34] HOST ALERT: galera2;DOWN;SOFT;1;connect to address galera2 and port 3306: Connection refused
[10-26-2026 04:32:06] HOST ALERT: presentation-lb-master;UP;SOFT;1;TCP OK - 0.001 second response time on presentation-lb-master port 80
[10-26-2026 04:31:11] HOST ALERT: presentation-lb-master;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: presentation-lb-master
[10-26-2026 04:26:25] SERVICE ALERT: application-lb-master;Application HTTP;OK;HARD;1;HTTP OK: HTTP/1.1 200 OK - 378 bytes in 0.005 second response time
[10-26-2026 04:18:57] HOST ALERT: application-lb-master;UP;SOFT;1;TCP OK - 0.001 second response time on application-lb-master port 8080
[10-26-2026 04:18:57] HOST FLAPPING ALERT: application-lb-master;STARTED; Host appears to have started flapping (22.9% change > 20.0% threshold)
[10-26-2026 04:17:57] HOST ALERT: application-lb-master;DOWN;SOFT;3;TCP CRITICAL - Invalid hostname, address or socket: application-lb-master
[10-26-2026 04:16:55] HOST ALERT: application-lb-master;DOWN;SOFT;2;TCP CRITICAL - Invalid hostname, address or socket: application-lb-master
[10-26-2026 04:16:31] SERVICE ALERT: application-lb-master;Application HTTP;UNKNOWN;HARD;1;check_http: Invalid hostname/address - application-lb-master
[10-26-2026 04:15:52] HOST ALERT: application-lb-master;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: application-lb-master
[10-26-2026 04:10:52] HOST ALERT: application-lb-master;UP;SOFT;1;TCP OK - 0.002 second response time on application-lb-master port 8080
[10-26-2026 04:09:52] HOST ALERT: application-lb-master;DOWN;SOFT;2;TCP CRITICAL - Invalid hostname, address or socket: application-lb-master
[10-26-2026 04:08:49] HOST ALERT: application-lb-master;DOWN;SOFT;1;TCP CRITICAL - Invalid hostname, address or socket: application-lb-master
October 26, 2026 01:00

Gaps the tests will likely expose

- `DB_HOST=galera1` in the app is a single point of failure.
- Redis and MongoDB are single instances.
- `presentation-entry` is a single point of failure.
- Compose does not restart a container that was stopped deliberately.`restart: unless-stopped`

Commonly used tools

Area	Tools
Carga HTTP	k6 (scripts JS, compatible con CI), Locust (Python), Gatling, wrk/wrk2 (tasa constante, latencia precisa), vegeta, hey, bombardier, Artillery
Carga empresarial / distribuida	BlazeMeter, Tsung, NBomber, Taurus (wrapper YAML sobre JMeter, Gatling, etc.)

Caos, contenedores/Docker	Pumba (matar, pausar, netem, estrés), Kit de herramientas del Caos (experimentos declarativos), Gremlin (SaaS)
Caos, Kubernetes	Malla Caos, LitmusChaos, Mono del Caos de Netflix / Ejército Simio (nube, Spinnaker)
Caos, nube	AWS Fault Injection Service, Azure Chaos Studio
Fallos de red	Toxiproxy, TC/Netem, Comcast, iptables
Estrés de huésped/recurso	Tensión-ng, fio, iperf3
Base de datos	sysbench, HammerDB, mysqlslap, pgbench, YCSB, mongoperf, redis-benchmark
Corrección distribuida	Jepsen (para bases de datos como Galera y MongoDB)
Observabilidad	Prometheus + Grafana, cAdvisor, node_exporter, Loki, Jaeger/OpenTelemetry, HAProxy estadísticas y exportador, mysqld_exporter, Nagios/Zabbix

Puerta de CI	Acciones de GitHub con umbrales k6, Taurus
Días de partido	Manuales escritos y autopsias sin culpa

Próximos pasos sugeridos

1. Añade Prometheus y Grafana (exportador HAProxy, cAdvisor) para que cada experimento tenga gráficos de su hipótesis.
2. Añade Pumba o Toxiproxy al laboratorio para experimentos de latencia, pérdida y partición.
3. Añade k6 o Locust para pruebas guionizadas basadas en umbrales que puedan ejecutarse en CI.
4. Elimina el punto único (proxy consciente de la base de datos o endpoint gestionado) y vuelve a ejecutar la prueba de conmutación por error de la base de datos.`galera1`